

MSA Day 1 실습 - 맛집 리뷰 서비스

Chapter 1: 독립 서비스 2개 만들기

실습 목표

오전에 배운 MSA 개념을 직접 적용하여, **맛집 리뷰 서비스**를 MSA로 구현합니다.

이번 챕터에서는 **서비스 2개를 각각 독립적으로** 만들고 동작을 확인합니다.

서비스 간 통신은 아직 구현하지 않습니다.

member-service (8081)

- 회원가입
- 회원 조회
- PostgreSQL (member_db)

review-service (8082)

- 리뷰 작성
- 리뷰 목록/상세
- 리뷰 수정/삭제
- PostgreSQL (review_db)

각각 독립적으로 실행 & 테스트

전체 흐름 미리보기

Step 1: PostgreSQL 데이터베이스 2개 생성

Step 2: member-service 프로젝트 생성 & 구현

Step 3: member-service 테스트 (Postman)

Step 4: review-service 프로젝트 생성 & 구현

Step 5: review-service 테스트 (Postman)

Step 1: PostgreSQL 데이터베이스 생성

터미널에서 PostgreSQL에 접속합니다.

```
psql postgres
```

데이터베이스 2개를 생성합니다.

```
CREATE DATABASE member_db;  
CREATE DATABASE review_db;
```

생성 확인:

```
\l
```

목록에 `member_db` 와 `review_db` 가 보이면 성공입니다.

나갈 때:

```
\q
```

Step 2: member-service 프로젝트 생성

Spring Initializr (start.spring.io)

```
Project: Gradle - Groovy  
Language: Java  
Spring Boot: 3.5.10  
Group: com.example  
Artifact: member-service  
Package name: com.example.memberservice  
Packaging: Jar  
Java: 17  
Configuration: YAML
```

Dependencies:

- ✓ Spring Web
- ✓ Spring Data JPA
- ✓ PostgreSQL Driver
- ✓ Lombok

Generate → 압축 해제 → IntelliJ에서 build.gradle 선택하여 Open as Project

Step 2-1: application.yaml 설정

src/main/resources/application.yaml

```
server:
  port: 8081

spring:
  application:
    name: member-service

  datasource:
    url: jdbc:postgresql://localhost:5432/member_db
    username: 본인_유저네임

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

| username은 본인의 PostgreSQL 사용자명을 입력하세요.

Step 2-2: Member Entity 작성

domain/Member.java

```
package com.example.memberservice.domain;

import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import java.time.LocalDateTime;
```

```

@Entity
@Table(name = "members")
@Getter
@NoArgsConstructor(access = AccessLevel.PROTECTED)
@AllArgsConstructor
@Builder
public class Member {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 50)
    private String nickname;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(nullable = false)
    private String password;

    @CreationTimestamp
    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @UpdateTimestamp
    @Column(nullable = false)
    private LocalDateTime updatedAt;
}

```

포인트:

- 오전의 `User` 대신 `Member` 라는 이름을 사용합니다.
- `username` 대신 `nickname` 을 사용합니다.

- 구조는 동일하지만 도메인 용어가 다릅니다.

Step 2-3: MemberRepository 작성

repository/MemberRepository.java

```
package com.example.memberservice.repository;

import com.example.memberservice.domain.Member;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import java.util.Optional;

@Repository
public interface MemberRepository extends JpaRepository<Member, Long> {

    Optional<Member> findByNickname(String nickname);

    boolean existsByNickname(String nickname);

    boolean existsByEmail(String email);
}
```

Step 2-4: MemberRequestDto 작성

dto/MemberRequestDto.java

```
package com.example.memberservice.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;
```

```

@Getter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class MemberRequestDto {

    private String nickname;
    private String email;
    private String password;
}

```

Step 2-5: MemberResponseDto 작성

dto/MemberResponseDto.java

```

package com.example.memberservice.dto;

import com.example.memberservice.domain.Member;
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Getter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class MemberResponseDto {

    private Long id;
    private String nickname;
    private String email;
    // password는 응답에 포함하지 않음 (보안)
}

```

```

        private LocalDateTime createdAt;

        public static MemberResponseDto from(Member member) {
            return MemberResponseDto.builder()
                .id(member.getId())
                .nickname(member.getNickname())
                .email(member.getEmail())
                .createdAt(member.getCreatedAt())
                .build();
        }
    }
}

```

Step 2-6: MemberService 작성

service/MemberService.java

```

package com.example.memberservice.service;

import com.example.memberservice.domain.Member;
import com.example.memberservice.dto.MemberRequestDto;
import com.example.memberservice.dto.MemberResponseDto;
import com.example.memberservice.repository.MemberRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class MemberService {

    private final MemberRepository memberRepository;

    // 회원가입

```

```

@Transactional
public MemberResponseDto createMember(MemberRequestDto request) {
    // 닉네임 중복 체크
    if (memberRepository.existsByNickname(request.getNickname())) {
        throw new IllegalArgumentException(
            "이미 존재하는 닉네임입니다.");
    }

    // 이메일 중복 체크
    if (memberRepository.existsByEmail(request.getEmail())) {
        throw new IllegalArgumentException(
            "이미 존재하는 이메일입니다.");
    }

    Member member = Member.builder()
        .nickname(request.getNickname())
        .email(request.getEmail())
        .password(request.getPassword())
        .build();

    Member saved = memberRepository.save(member);

    return MemberResponseDto.from(saved);
}

// 회원 조회 (ID)
public MemberResponseDto getMember(Long id) {
    Member member = memberRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentException(
            "회원을 찾을 수 없습니다."));

    return MemberResponseDto.from(member);
}

```



```

    }

    // 회원 조회 (닉네임)
    public MemberResponseDto getMemberByNickname(String nickname) {
        Member member = memberRepository.findByNickname(nickname)
            .orElseThrow(() -> new IllegalArgumentException(
                "회원을 찾을 수 없습니다."));

        return MemberResponseDto.from(member);
    }
}

```

Step 2-7: MemberController 작성

controller/MemberController.java

```

package com.example.memberservice.controller;

import com.example.memberservice.dto.MemberRequestDto;
import com.example.memberservice.dto.MemberResponseDto;
import com.example.memberservice.service.MemberService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/members")
@RequiredArgsConstructor
public class MemberController {

    private final MemberService memberService;
}

```

```

// 회원가입
@PostMapping
public ResponseEntity<MemberResponseDto> createMember(
    @RequestBody MemberRequestDto request) {
    MemberResponseDto response =
        memberService.createMember(request);
    return ResponseEntity
        .status(HttpStatus.CREATED).body(response);
}

// 회원 조회 (ID)
@GetMapping("/{id}")
public ResponseEntity<MemberResponseDto> getMember(
    @PathVariable Long id) {
    MemberResponseDto response =
        memberService.getMember(id);
    return ResponseEntity.ok(response);
}

// 회원 조회 (닉네임)
@GetMapping
public ResponseEntity<MemberResponseDto> getMemberByNickname(
    @RequestParam String nickname) {
    MemberResponseDto response =
        memberService.getMemberByNickname(nickname);
    return ResponseEntity.ok(response);
}
}

```

Step 2-8: 패키지 구조 확인

작성이 끝났으면 패키지 구조를 확인합니다.

```
com.example.memberservice
├── MemberServiceApplication.java
├── controller
│   └── MemberController.java
├── domain
│   └── Member.java
├── dto
│   ├── MemberRequestDto.java
│   └── MemberResponseDto.java
├── repository
│   └── MemberRepository.java
└── service
    └── MemberService.java
```

Step 3: member-service 실행 & 테스트

실행

MemberServiceApplication을 실행합니다.

콘솔에서 확인할 것:

- Tomcat started on port(s): 8081 ← 포트 확인
- Hibernate: create table members ← 테이블 생성 확인

Step 3-1: 회원가입 테스트

Postman:

POST http://localhost:8081/api/members

Headers:

Content-Type: application/json

Body (raw JSON):

```
{
  "nickname": "맛집헌터",
  "email": "hunter@example.com",
  "password": "1234"
}
```

기대 응답 (201 Created):

```
{
  "id": 1,
  "nickname": "맛집헌터",
  "email": "hunter@example.com",
  "createdAt": "2026-02-18T..."
}
```

| password가 응답에 포함되지 않는 것을 확인하세요.

Step 3-2: 회원 조회 테스트 (ID)

```
GET http://localhost:8081/api/members/1
```

기대 응답 (200 OK):

```
{
  "id": 1,
  "nickname": "맛집헌터",
  "email": "hunter@example.com",
  "createdAt": "2026-02-18T..."
}
```

Step 3-3: 회원 조회 테스트 (닉네임)

```
GET http://localhost:8081/api/members?nickname=맛집헌터
```

Step 3-4: 중복 체크 테스트

같은 닉네임으로 다시 회원가입:

```
POST http://localhost:8081/api/members
```

Body:

```
{  
  "nickname": "맛집헌터",  
  "email": "other@example.com",  
  "password": "5678"  
}
```

기대 결과: 에러 응답 (닉네임 중복)

Step 3-5: 회원 추가 등록

Chapter 2에서 리뷰 작성에 사용할 회원을 미리 등록합니다.

```
POST http://localhost:8081/api/members
```

Body:

```
{  
  "nickname": "먹방크리에이터",  
  "email": "creator@example.com",  
  "password": "1234"  
}
```

| 이 회원의 id를 기억해두세요. (아마 id = 2)

 **member-service 체크포인트**

다음으로 넘어가기 전에 확인하세요:

- ☐ 앱이 8081 포트에서 실행되는가?
- ☐ 회원가입이 정상 동작하는가?
- ☐ 회원 조회(ID, 닉네임)가 정상 동작하는가?
- ☐ 중복 닉네임으로 가입 시 에러가 발생하는가?
- ☐ 회원이 2명 이상 등록되어 있는가?

모두 확인했으면 review-service로 넘어갑니다.

Step 4: review-service 프로젝트 생성

Spring Initializr (start.spring.io)

```
Project: Gradle - Groovy
Language: Java
Spring Boot: 3.5.10
Group: com.example
Artifact: review-service
Package name: com.example.reviewservice
Packaging: Jar
Java: 17
Configuration: YAML
```

Dependencies:

- ☒ Spring Web
- ☒ Spring Data JPA
- ☒ PostgreSQL Driver
- ☒ Lombok
- ☒ OpenFeign ← 미리 추가

OpenFeign은 Chapter 2에서 사용합니다. 미리 추가만 해둡니다.

Generate → 압축 해제 → IntelliJ에서 Open as Project

Step 4-1: application.yaml 설정

src/main/resources/application.yaml

```
server:
  port: 8082

spring:
  application:
    name: review-service

  datasource:
    url: jdbc:postgresql://localhost:5432/review_db
    username: 본인_유저네임

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

Step 4-2: Review Entity 작성

domain/Review.java

```
package com.example.reviewservice.domain;

import jakarta.persistence.*;
import lombok.*;
import org.hibernate.annotations.CreationTimestamp;
import org.hibernate.annotations.UpdateTimestamp;

import java.time.LocalDateTime;

@Entity
@Table(name = "reviews")
@Getter
```

```

@NoArgsConstructor(access = AccessLevel.PROTECTED)
@AllArgsConstructor
@Builder
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String storeName;          // 가게 이름

    @Column(nullable = false)
    private Integer rating;            // 별점 (1~5)

    @Column(nullable = false, columnDefinition = "TEXT")
    private String content;           // 리뷰 내용

    @Column(nullable = false)
    private Long memberId;            // member-service의 Member ID

    @CreationTimestamp
    @Column(nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @UpdateTimestamp
    @Column(nullable = false)
    private LocalDateTime updatedAt;

    // 리뷰 수정
    public void update(String storeName, Integer rating,
                      String content) {
        this.storeName = storeName;
        this.rating = rating;
        this.content = content;
    }

```



```
}  
}
```

포인트:

- `memberId` 필드로 작성자를 참조합니다 (FK 아님. 다른 DB에 있으니까)
- `storeName` 으로 어떤 가게의 리뷰인지 저장합니다
- `rating` 은 1~5 별점입니다

Step 4-3: ReviewRepository 작성

`repository/ReviewRepository.java`

```
package com.example.reviewservice.repository;  
  
import com.example.reviewservice.domain.Review;  
import org.springframework.data.jpa.repository.JpaRepository;  
import org.springframework.stereotype.Repository;  
  
import java.util.List;  
  
@Repository  
public interface ReviewRepository  
    extends JpaRepository<Review, Long> {  
  
    // 특정 회원의 리뷰 조회  
    List<Review> findByMemberId(Long memberId);  
  
    // 가게 이름으로 리뷰 검색  
    List<Review> findByStoreNameContaining(String storeName);  
}
```

Step 4-4: ReviewRequestDto 작성

`dto/ReviewRequestDto.java`

```

package com.example.reviewservice.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;

@Getter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ReviewRequestDto {

    private String storeName;
    private Integer rating;
    private String content;
    private Long memberId;
}

```

이 단계에서는 ResponseDto를 만들지 않습니다.

Review Entity를 그대로 반환합니다.

Chapter 2에서 작성자 정보를 포함한 ResponseDto를 만들 예정입니다.

Step 4-5: ReviewService 작성

`service/ReviewService.java`

```

package com.example.reviewservice.service;

import com.example.reviewservice.domain.Review;
import com.example.reviewservice.dto.ReviewRequestDto;
import com.example.reviewservice.repository.ReviewRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transaction

```

```

al;

import java.util.List;

@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class ReviewService {

    private final ReviewRepository reviewRepository;

    // 리뷰 작성
    @Transactional
    public Review createReview(ReviewRequestDto request) {
        Review review = Review.builder()
            .storeName(request.getStoreName())
            .rating(request.getRating())
            .content(request.getContent())
            .memberId(request.getMemberId())
            .build();

        return reviewRepository.save(review);
    }

    // 리뷰 목록 조회
    public List<Review> getAllReviews() {
        return reviewRepository.findAll();
    }

    // 리뷰 상세 조회
    public Review getReview(Long id) {
        return reviewRepository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException(
on(
                "리뷰를 찾을 수 없습니다."));
    }

```

```

// 리뷰 수정
@Transactional
public Review updateReview(Long id,
                           ReviewRequestDto request) {
    Review review = reviewRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentExceptionExcepti
on(
        "리뷰를 찾을 수 없습니다."));

    review.update(
        request.getStoreName(),
        request.getRating(),
        request.getContent()
    );

    return review;
}

// 리뷰 삭제
@Transactional
public void deleteReview(Long id) {
    Review review = reviewRepository.findById(id)
        .orElseThrow(() -> new IllegalArgumentExceptionExcepti
on(
        "리뷰를 찾을 수 없습니다."));

    reviewRepository.delete(review);
}
}

```

Step 4-6: ReviewController 작성

controller/ReviewController.java

```

package com.example.reviewservice.controller;

import com.example.reviewservice.domain.Review;
import com.example.reviewservice.dto.ReviewRequestDto;
import com.example.reviewservice.service.ReviewService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/reviews")
@RequiredArgsConstructor
public class ReviewController {

    private final ReviewService reviewService;

    // 리뷰 작성
    @PostMapping
    public ResponseEntity<Review> createReview(
        @RequestBody ReviewRequestDto request) {
        Review saved = reviewService.createReview(request);
        return ResponseEntity
            .status(HttpStatus.CREATED).body(saved);
    }

    // 리뷰 목록 조회
    @GetMapping
    public ResponseEntity<List<Review>> getAllReviews() {
        List<Review> reviews = reviewService.getAllReviews();
        return ResponseEntity.ok(reviews);
    }
}

```

```

// 리뷰 상세 조회
@GetMapping("/{id}")
public ResponseEntity<Review> getReview(
    @PathVariable Long id) {
    Review review = reviewService.getReview(id);
    return ResponseEntity.ok(review);
}

// 리뷰 수정
@PutMapping("/{id}")
public ResponseEntity<Review> updateReview(
    @PathVariable Long id,
    @RequestBody ReviewRequestDto request) {
    Review updated =
        reviewService.updateReview(id, request);
    return ResponseEntity.ok(updated);
}

// 리뷰 삭제
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteReview(
    @PathVariable Long id) {
    reviewService.deleteReview(id);
    return ResponseEntity.noContent().build();
}
}

```

Step 4-7: 패키지 구조 확인

```

com.example.reviewservice
├─ ReviewServiceApplication.java
├─ controller
│   └─ ReviewController.java
└─ domain

```

```
|   └─ Review.java
|─ dto
|   └─ ReviewRequestDto.java
|─ repository
|   └─ ReviewRepository.java
└─ service
    └─ ReviewService.java
```

Step 5: review-service 실행 & 테스트

실행

ReviewServiceApplication을 실행합니다.

| member-service(8081)도 계속 실행 중인 상태로 두세요.

콘솔에서 확인할 것:

- Tomcat started on port(s): 8082 ← 포트 확인
- Hibernate: create table reviews ← 테이블 생성 확인

Step 5-1: 리뷰 작성 테스트

Postman:

POST http://localhost:8082/api/reviews

Headers:

Content-Type: application/json

Body (raw JSON):

```
{
  "storeName": "역전우동",
  "rating": 5,
  "content": "우동이 정말 맛있어요! 국물이 진하고 면도 쫄깃!",
}
```

```
"memberId": 1
}
```

기대 응답 (201 Created):

```
{
  "id": 1,
  "storeName": "역전우동",
  "rating": 5,
  "content": "우동이 정말 맛있어요! 국물이 진하고 면도 쫄깃!",
  "memberId": 1,
  "createdAt": "2026-02-18T...",
  "updatedAt": "2026-02-18T..."
}
```

응답에 `memberId` 는 있지만 누가 작성했는지(닉네임) 알 수 없습니다.
→ Chapter 2에서 OpenFeign으로 해결합니다.

Step 5-2: 리뷰 추가 작성

다른 회원으로도 리뷰를 작성해봅니다.

POST `http://localhost:8082/api/reviews`

Body:

```
{
  "storeName": "홍콩반점",
  "rating": 4,
  "content": "짬뽕이 얼큰하니 좋습니다. 양도 많아요.",
  "memberId": 2
}
```

POST `http://localhost:8082/api/reviews`

Body:

```
{
  "storeName": "역전우동",
  "rating": 4,
  "content": "가성비 좋은 우동집. 튀김도 바삭해요.",
  "memberId": 2
}
```

Step 5-3: 리뷰 목록 조회 테스트

GET http://localhost:8082/api/reviews

기대 결과: 작성한 리뷰 3개가 모두 조회됩니다.

Step 5-4: 리뷰 상세 조회 테스트

GET http://localhost:8082/api/reviews/1

Step 5-5: 리뷰 수정 테스트

PUT http://localhost:8082/api/reviews/1

Body:

```
{
  "storeName": "역전우동 강남점",
  "rating": 5,
  "content": "재방문! 역시 맛있다. 강남점도 굿!",
  "memberId": 1
}
```

확인: storeName, rating, content가 변경되었는지 확인합니다.

Step 5-6: 리뷰 삭제 테스트

```
DELETE http://localhost:8082/api/reviews/3
```

기대 결과: 204 No Content

삭제 후 목록 조회로 확인:

```
GET http://localhost:8082/api/reviews
```

✓ Chapter 1 체크포인트

모든 항목을 확인하세요:

- ☐ member-service가 8081에서 정상 실행
- ☐ review-service가 8082에서 정상 실행
- ☐ 회원가입 + 조회 정상 동작
- ☐ 리뷰 CRUD(작성/목록/상세/수정/삭제) 정상 동작
- ☐ 회원이 2명 이상 등록됨
- ☐ 리뷰가 2개 이상 등록됨



현재 상태와 한계

지금까지 2개의 서비스가 **독립적으로** 동작하는 것을 확인했습니다.

하지만 한 가지 아쉬운 점이 있습니다:

```
// 리뷰 조회 응답
{
  "id": 1,
  "storeName": "역전우동",
  "rating": 5,
  "content": "우동이 정말 맛있어요!",
  "memberId": 1,           ← 누구인지 모름
```

```
"createdAt": "..."  
}
```

`memberId: 1` 만으로는 누가 작성한 리뷰인지 알 수 없습니다.

Chapter 2에서는 OpenFeign을 사용하여 member-service를 호출하고, 리뷰 응답에 **작성자 닉네임**을 포함시킵니다.

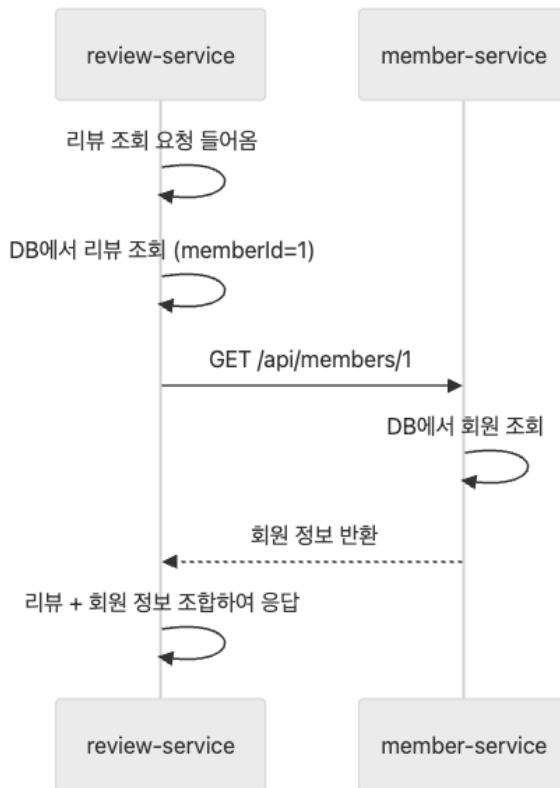
```
// Chapter 2 완성 후 기대 응답  
{  
  "id": 1,  
  "storeName": "역전우동",  
  "rating": 5,  
  "content": "우동이 정말 맛있어요!",  
  "memberId": 1,  
  "nickname": "맛집헌터",    ← 작성자 정보 추가  
  "createdAt": "..."  
}
```

Chapter 2: 서비스 간 통신 (OpenFeign)

실습 목표

Chapter 1에서 만든 두 서비스를 **OpenFeign**으로 연결합니다.

리뷰를 조회할 때 member-service를 호출하여 **작성자 닉네임**을 함께 반환합니다.



📌 전체 흐름 미리보기

- Step 1: build.gradle 확인 (OpenFeign 의존성)
- Step 2: @EnableFeignClients 설정
- Step 3: MemberDto 생성 (member-service 응답 받을 DTO)
- Step 4: MemberServiceClient 생성 (Feign 인터페이스)
- Step 5: ReviewResponseDto 생성 (작성자 정보 포함)
- Step 6: ReviewService 수정 (OpenFeign 호출 추가)
- Step 7: ReviewController 수정 (반환 타입 변경)
- Step 8: 테스트

Step 1: build.gradle 확인

review-service의 `build.gradle` 을 열어 OpenFeign 의존성이 있는지 확인합니다.

Chapter 1에서 프로젝트 생성 시 추가했으므로 이미 있어야 합니다.

```
dependencies {  
    implementation 'org.springframework.cloud:spring-cloud-starter-openfeign'  
    // ... 나머지 의존성들  
}
```

또한 `dependencyManagement` 블록에 Spring Cloud BOM이 있는지 확인합니다.

```
ext {  
    set('springCloudVersion', "2025.0.1")  
}  
  
dependencyManagement {  
    imports {  
        mavenBom "org.springframework.cloud:spring-cloud-dependencies:${springCloudVersion}"  
    }  
}
```

만약 없다면 위 내용을 추가하세요.

Step 2: @EnableFeignClients 설정

`ReviewServiceApplication.java` 를 수정합니다.

```
package com.example.reviewservice;  
  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
import org.springframework.cloud.openfeign.EnableFeignClients;  
  
@EnableFeignClients  
@SpringBootApplication  
public class ReviewServiceApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(ReviewServiceApplication.class, args);  
    }  
}
```

```

@SpringBootApplication
@EnableFeignClients      // ← 추가
public class ReviewServiceApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            ReviewServiceApplication.class, args);
    }
}

```

`@EnableFeignClients` 가 하는 일:

Spring Boot에게 "이 프로젝트에서 `@FeignClient` 가 붙은 인터페이스를 찾아서 Bean으로 등록해줘"라고 알려줍니다.

오전에 배운 내용 복습

Spring Data JPA가 Repository 인터페이스의 구현체를 자동 생성하듯,
OpenFeign도 FeignClient 인터페이스의 구현체를 자동 생성합니다.

Step 3: MemberDto 생성

member-service의 응답을 받을 DTO를 review-service에 만듭니다.

`dto/MemberDto.java`

```

package com.example.reviewservice.dto;

import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Getter;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Getter
@NoArgsConstructor
@AllArgsConstructor

```

```
@Builder
public class MemberDto {

    private Long id;
    private String nickname;
    private String email;
    private LocalDateTime createdAt;
}
```

포인트:

- member-service의 `MemberResponseDto` 와 같은 필드 구조입니다.
- review-service는 member-service의 코드를 모릅니다.
JSON 응답 형태만 맞추면 됩니다.

Step 4: MemberServiceClient 생성 (Feign 인터페이스)

새 패키지 `client` 를 만들고 인터페이스를 작성합니다.

`client/MemberServiceClient.java`

```
package com.example.reviewservice.client;

import com.example.reviewservice.dto.MemberDto;
import org.springframework.cloud.openfeign.FeignClient;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;

@FeignClient(
    name = "member-service",
    url = "http://localhost:8081"
)
public interface MemberServiceClient {

    @GetMapping("/api/members/{id}")
```

```
MemberDto getMember(@PathVariable("id") Long id);
}
```

코드 해석:

부분	의미
@FeignClient	이 인터페이스가 HTTP 클라이언트임을 선언
name = "member-service"	서비스 식별자 (Eureka 사용 시 이 이름으로 찾음)
url = "http://localhost:8081"	실제 호출할 주소
@GetMapping("/api/members/{id}")	member-service의 API와 동일하게 작성
MemberDto	응답을 자동으로 이 타입으로 변환

내부적으로 일어나는 일:

```

userClient.getMember(1) 호출
    ↓
OpenFeign이 HTTP 요청 생성: GET http://localhost:8081/api/members/1
    ↓
member-service가 JSON 응답 반환
    ↓
OpenFeign이 JSON → MemberDto로 변환
    ↓
MemberDto 객체 반환

```

Step 5: ReviewResponseDto 생성

이제 리뷰 + 작성자 정보를 함께 반환할 DTO를 만듭니다.

dto/ReviewResponseDto.java

```

package com.example.reviewservice.dto;

import com.example.reviewservice.domain.Review;
import lombok.AllArgsConstructor;
import lombok.Builder;

```



```

import lombok.Getter;
import lombok.NoArgsConstructor;

import java.time.LocalDateTime;

@Getter
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class ReviewResponseDto {

    private Long id;
    private String storeName;
    private Integer rating;
    private String content;
    private Long memberId;
    private String nickname;          // ← 작성자 닉네임 추가
    private LocalDateTime createdAt;
    private LocalDateTime updatedAt;

    // Review Entity만으로 변환 (작성자 정보 없음)
    public static ReviewResponseDto from(Review review) {
        return ReviewResponseDto.builder()
            .id(review.getId())
            .storeName(review.getStoreName())
            .rating(review.getRating())
            .content(review.getContent())
            .memberId(review.getMemberId())
            .createdAt(review.getCreatedAt())
            .updatedAt(review.getUpdatedAt())
            .build();
    }

    // Review + Member 정보로 변환 (작성자 정보 포함)
    public static ReviewResponseDto from(Review review,
                                         MemberDto member) {

```

```

        return ReviewResponseDto.builder()
            .id(review.getId())
            .storeName(review.getStoreName())
            .rating(review.getRating())
            .content(review.getContent())
            .memberId(review.getMemberId())
            .nickname(member.getNickname()) // ← 추가
            .createdAt(review.getCreatedAt())
            .updatedAt(review.getUpdatedAt())
            .build();
    }
}

```

포인트:

- `from` 메서드가 **2개**입니다 (메서드 오버로딩)
 - `from(Review)` → 작성자 정보 없이 변환
 - `from(Review, MemberDto)` → 작성자 정보 포함 변환

Step 6: ReviewService 수정

OpenFeign을 통해 member-service를 호출하도록 수정합니다.

`service/ReviewService.java` 전체를 아래로 교체합니다.

```

package com.example.reviewservice.service;

import com.example.reviewservice.client.MemberServiceClient;
import com.example.reviewservice.domain.Review;
import com.example.reviewservice.dto.MemberDto;
import com.example.reviewservice.dto.ReviewRequestDto;
import com.example.reviewservice.dto.ReviewResponseDto;
import com.example.reviewservice.repository.ReviewRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

```

```

al;

import java.util.List;
import java.util.stream.Collectors;

@Service
@RequiredArgsConstructor
@Transactional(readOnly = true)
public class ReviewService {

    private final ReviewRepository reviewRepository;
    private final MemberServiceClient memberServiceClient;
    // ↑ OpenFeign 주입 (일반 Bean처럼 사용)

    // 리뷰 작성
    @Transactional
    public ReviewResponseDto createReview(
        ReviewRequestDto request) {

        Review review = Review.builder()
            .storeName(request.getStoreName())
            .rating(request.getRating())
            .content(request.getContent())
            .memberId(request.getMemberId())
            .build();

        Review saved = reviewRepository.save(review);

        // member-service에서 작성자 정보 가져오기
        MemberDto member =
            memberServiceClient.getMember(saved.getMemberId
        ());

        return ReviewResponseDto.from(saved, member);
    }
}

```

```

// 리뷰 목록 조회
public List<ReviewResponseDto> getAllReviews() {
    List<Review> reviews = reviewRepository.findAll();

    return reviews.stream()
        .map(review -> {
            // 각 리뷰마다 작성자 정보 조회
            MemberDto member = memberServiceClient
                .getMember(review.getMemberId());
            return ReviewResponseDto
                .from(review, member);
        })
        .collect(Collectors.toList());
}

// 리뷰 상세 조회
public ReviewResponseDto getReview(Long id) {
    Review review = reviewRepository.findById(id)
        .orElseThrow(() ->
            new IllegalArgumentException(
                "리뷰를 찾을 수 없습니다."));

    // 작성자 정보 가져오기
    MemberDto member = memberServiceClient
        .getMember(review.getMemberId());

    return ReviewResponseDto.from(review, member);
}

// 리뷰 수정
@Transactional
public ReviewResponseDto updateReview(
    Long id, ReviewRequestDto request) {

    Review review = reviewRepository.findById(id)
        .orElseThrow(() ->

```

```

        new IllegalArgumentException(
            "리뷰를 찾을 수 없습니다."));

    review.update(
        request.getStoreName(),
        request.getRating(),
        request.getContent()
    );

    MemberDto member = memberServiceClient
        .getMember(review.getMemberId());

    return ReviewResponseDto.from(review, member);
}

// 리뷰 삭제
@Transactional
public void deleteReview(Long id) {
    Review review = reviewRepository.findById(id)
        .orElseThrow(() ->
            new IllegalArgumentException(
                "리뷰를 찾을 수 없습니다."));

    reviewRepository.delete(review);
}
}

```

Chapter 1과 비교 - 변경된 부분:

변경점	Before (Chapter 1)	After (Chapter 2)
의존성	ReviewRepository만	+ MemberServiceClient 추가
반환 타입	Review (Entity)	ReviewResponseDto
작성자 정보	없음	OpenFeign으로 조회하여 포함

Step 7: ReviewController 수정

반환 타입을 `Review` → `ReviewResponseDto` 로 변경합니다.

`controller/ReviewController.java` 전체를 아래로 교체합니다.

```
package com.example.reviewservice.controller;

import com.example.reviewservice.dto.ReviewRequestDto;
import com.example.reviewservice.dto.ReviewResponseDto;
import com.example.reviewservice.service.ReviewService;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/reviews")
@RequiredArgsConstructor
public class ReviewController {

    private final ReviewService reviewService;

    // 리뷰 작성
    @PostMapping
    public ResponseEntity<ReviewResponseDto> createReview(
        @RequestBody ReviewRequestDto request) {
        ReviewResponseDto response =
            reviewService.createReview(request);
        return ResponseEntity
            .status(HttpStatus.CREATED).body(response);
    }

    // 리뷰 목록 조회
    @GetMapping
```

```

public ResponseEntity<List<ReviewResponseDto>>
    getAllReviews() {
        List<ReviewResponseDto> response =
            reviewService.getAllReviews();
        return ResponseEntity.ok(response);
    }

// 리뷰 상세 조회
@GetMapping("/{id}")
public ResponseEntity<ReviewResponseDto> getReview(
    @PathVariable Long id) {
    ReviewResponseDto response =
        reviewService.getReview(id);
    return ResponseEntity.ok(response);
}

// 리뷰 수정
@PutMapping("/{id}")
public ResponseEntity<ReviewResponseDto> updateReview(
    @PathVariable Long id,
    @RequestBody ReviewRequestDto request) {
    ReviewResponseDto response =
        reviewService.updateReview(id, request);
    return ResponseEntity.ok(response);
}

// 리뷰 삭제
@DeleteMapping("/{id}")
public ResponseEntity<Void> deleteReview(
    @PathVariable Long id) {
    reviewService.deleteReview(id);
    return ResponseEntity.noContent().build();
}
}

```

Step 7-1: 패키지 구조 확인

```
com.example.reviewservice
├─ ReviewServiceApplication.java    ← @EnableFeignClients 추가됨
├─ client
│   └─ MemberServiceClient.java    ← 새로 추가
├─ controller
│   └─ ReviewController.java       ← 반환 타입 변경됨
├─ domain
│   └─ Review.java
├─ dto
│   ├── MemberDto.java             ← 새로 추가
│   ├── ReviewRequestDto.java
│   └─ ReviewResponseDto.java      ← 새로 추가
├─ repository
│   └─ ReviewRepository.java
└─ service
    └─ ReviewService.java          ← OpenFeign 호출 추가됨
```

Step 8: 테스트

실행 확인

두 서비스 모두 실행 중인지 확인하세요

- member-service → 8081
- review-service → 8082 (재시작 필요)

Step 8-1: 리뷰 상세 조회

Chapter 1에서 작성한 리뷰를 다시 조회합니다.

```
GET http://localhost:8082/api/reviews/1
```


Chapter 1 응답 (Before):

```
{
  "id": 1,
  "storeName": "역전우동",
  "rating": 5,
  "content": "우동이 정말 맛있어요! 국물이 진하고 면도 쫄깃!",
  "memberId": 1,
  "createdAt": "...",
  "updatedAt": "..."
}
```

Chapter 2 응답 (After):

```
{
  "id": 1,
  "storeName": "역전우동",
  "rating": 5,
  "content": "우동이 정말 맛있어요! 국물이 진하고 면도 쫄깃!",
  "memberId": 1,
  "nickname": "맛집헌터",
  "createdAt": "...",
  "updatedAt": "..."
}
```

"nickname": "맛집헌터" 가 추가된 것을 확인하세요.
review-service가 member-service를 호출하여 가져온 정보입니다.

Step 8-2: 리뷰 목록 조회

```
GET http://localhost:8082/api/reviews
```

기대 결과: 모든 리뷰에 작성자 닉네임이 포함됩니다.

```
[
  {
    "id": 1,
    "storeName": "역전우동 강남점",
    "rating": 5,
    "nickname": "맛집헌터",
    ...
  },
  {
    "id": 2,
    "storeName": "홍콩반점",
    "rating": 4,
    "nickname": "먹방크리에이터",
    ...
  }
]
```

Step 8-3: 새 리뷰 작성 테스트

POST http://localhost:8082/api/reviews

Body:

```
{
  "storeName": "스시히로",
  "rating": 5,
  "content": "오마카세 코스가 훌륭합니다. 샤리가 특히 좋아요.",
  "memberId": 1
}
```

기대 결과 (201 Created):

```
{
  "id": 3,
  "storeName": "스시히로",
```

```
"rating": 5,
"content": "오마카세 코스가 훌륭합니다. 샤리가 특히 좋아요.",
"memberId": 1,
"nickname": "맛집헌터",
"createdAt": "...",
"updatedAt": "..."
}
```

작성과 동시에 작성자 닉네임이 응답에 포함됩니다.

Step 8-4: member-service 중지 후 테스트

OpenFeign의 동작을 확실히 이해하기 위한 실험입니다.

1. member-service(8081)를 **중지**합니다.
2. 리뷰를 조회합니다.

```
GET http://localhost:8082/api/reviews/1
```

기대 결과: 에러 발생

```
500 Internal Server Error
```

왜 에러가 날까?

review-service가 member-service를 호출하는데,
member-service가 꺼져있으니 응답을 받을 수 없습니다.

이것이 MSA에서 **서비스 간 의존성**의 단점입니다.

오전에 배운 "MSA 단점: 네트워크 레이턴시, 장애 전파"를 직접 경험한 것입니다.

1. 다시 member-service를 **실행**합니다.
2. 동일한 요청을 다시 보냅니다 → 정상 응답 확인

Step 8-5: review-service 콘솔 로그 확인

리뷰를 조회할 때 review-service의 콘솔을 관찰합니다.

```
GET http://localhost:8082/api/reviews
```

콘솔에서 보이는 것:

```
Hibernate: select r1_0.id, ... from reviews r1_0
```

→ review_db에서 리뷰를 조회하는 SQL만 보입니다.

→ member-service 호출은 SQL이 아니라 **HTTP 요청**이므로 콘솔에 안 보입니다.

HTTP 호출 로그도 보고 싶다면 `application.yaml` 에 아래를 추가합니다:

```
logging:
  level:
    com.example.reviewservice: DEBUG
```

그리고 Feign 로그 설정 클래스를 추가합니다.

`config/FeignConfig.java`

```
package com.example.reviewservice.config;

import feign.Logger;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;

@Configuration
public class FeignConfig {

    @Bean
    Logger.Level feignLogLevel() {
        return Logger.Level.FULL;
    }
}
```

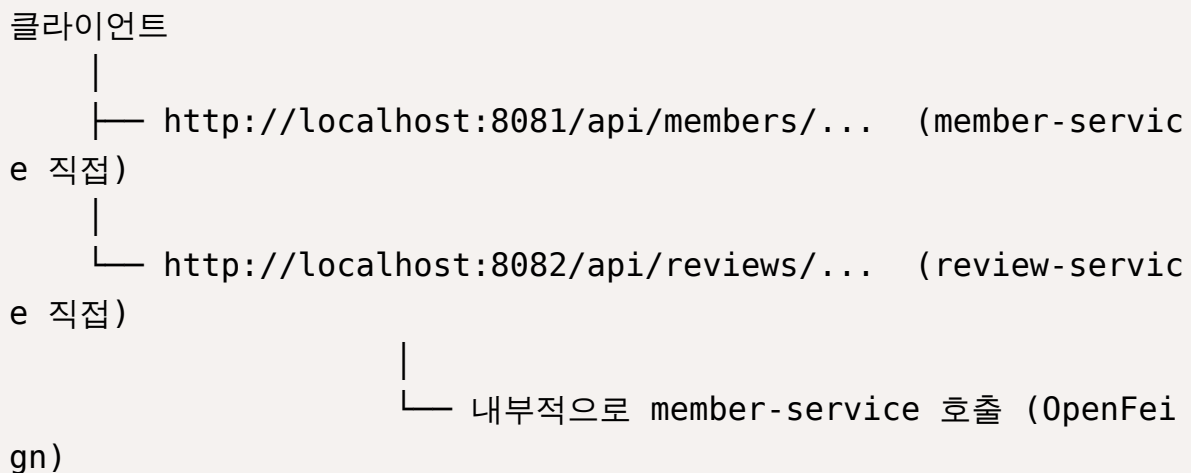
재시작 후 리뷰를 조회하면 콘솔에 HTTP 호출 로그가 출력됩니다:

```
[MemberServiceClient#getMember] ---> GET http://localhost:8081/api/members/1 HTTP/1.1
[MemberServiceClient#getMember] <--- HTTP/1.1 200 (15ms)
```

✓ Chapter 2 체크포인트

- ☐ review-service에 @EnableFeignClients가 추가되었는가?
- ☐ MemberServiceClient 인터페이스가 작성되었는가?
- ☐ 리뷰 조회 시 nickname이 응답에 포함되는가?
- ☐ 리뷰 목록 조회 시 모든 리뷰에 nickname이 있는가?
- ☐ member-service 중지 시 에러가 발생하는가?
- ☐ member-service 재시작 후 정상 동작하는가?

💡 현재 상태



지금은 클라이언트가 **각 서비스의 포트를 직접 알아야** 합니다.

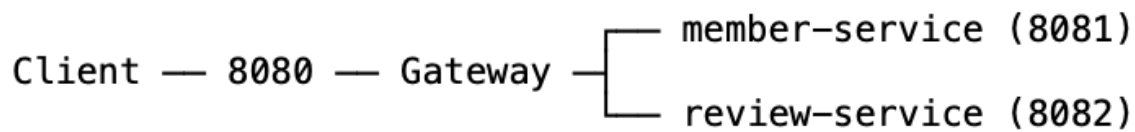
서비스가 2개일 때는 괜찮지만, 10개, 20개가 되면?
클라이언트가 모든 서비스의 주소를 관리해야 합니다.

Chapter 3에서는 API Gateway를 추가하여 **하나의 진입점(8080)**으로 모든 서비스에 접근할 수 있게 만듭니다.

Chapter 3: API Gateway + CORS

실습 목표

API Gateway를 추가하여 **하나의 진입점(8080)**으로 모든 서비스에 접근합니다.
CORS 설정을 추가하여 **프론트엔드에서 호출 가능**하도록 만듭니다.



`/api/members/**` → member-service
`/api/reviews/**` → review-service

전체 흐름 미리보기

- Step 1: api-gateway 프로젝트 생성
- Step 2: application.yaml 라우팅 설정
- Step 3: Gateway를 통한 서비스 호출 테스트
- Step 4: CORS 설정 추가
- Step 5: CORS 동작 확인 (브라우저 테스트)

Step 1: api-gateway 프로젝트 생성

Spring Initializr (start.spring.io)

Project: Gradle - Groovy
Language: Java
Spring Boot: 3.5.10
Group: com.example
Artifact: api-gateway

```
Package name: com.example.apigateway
Packaging: Jar
Java: 17
Configuration: YAML
```

Dependencies:

- ✓ Reactive Gateway
- ✓ Lombok

주의

- **Reactive Gateway** 를 선택합니다. (**Gateway** 로 검색하면 나옵니다)
- **Spring Web**은 추가하지 마세요. Gateway와 충돌합니다.

Generate → 압축 해제 → IntelliJ에서 Open as Project

Step 1-1: 왜 Reactive Gateway인가?

오전 강의에서 배운 내용 복습입니다.

API Gateway는 **요청을 받아서 다른 서비스로 전달하고, 응답을 돌려주는** 중계 역할입니다.
대부분의 시간을 **다른 서비스의 응답을 기다리는 데** 사용합니다.

리액티브(비동기)가 이런 I/O 대기 작업에 효율적이기 때문에 Gateway에 적합합니다.

Gateway만 리액티브이고, member-service와 review-service는
기존 MVC(동기) 방식 그대로입니다.
이것이 MSA의 장점 — 각 서비스가 독립적으로 기술을 선택할 수 있습니다.

Step 2: application.yaml 설정

```
src/main/resources/application.yaml
```

```
server:
  port: 8080

spring:
  application:
```

```

name: api-gateway

cloud:
  gateway:
    server:
      webflux:
        routes:
          # member-service 라우팅
          - id: member-service
            uri: http://localhost:8081
            predicates:
              - Path=/api/members/**

          # review-service 라우팅
          - id: review-service
            uri: http://localhost:8082
            predicates:
              - Path=/api/reviews/**

```

설정 해석:

항목	의미
<code>id: member-service</code>	이 라우팅 규칙의 이름 (구분용)
<code>uri: http://localhost:8081</code>	요청을 전달할 서비스 주소
<code>Path=/api/members/**</code>	이 경로로 들어온 요청을 해당 서비스로 전달

라우팅 동작:

```

클라이언트 → GET http://localhost:8080/api/members/1
Gateway    → Path가 /api/members/** 에 매칭
            → http://localhost:8081/api/members/1 로 전달

클라이언트 → GET http://localhost:8080/api/reviews/1
Gateway    → Path가 /api/reviews/** 에 매칭
            → http://localhost:8082/api/reviews/1 로 전달

```


버전 주의

인터넷에서 검색하면 아래와 같은 설정을 많이 볼 수 있습니다:

```
spring.cloud.gateway.routes ← 옛날 버전
```

Spring Cloud 2025.0.x 에서는 경로가 바뀌었습니다:

```
spring.cloud.gateway.server.webflux.routes ← 현재 버전
```

검색 시 주의

Step 3: Gateway를 통한 서비스 호출 테스트

3개 서비스 모두 실행

서비스	포트	확인
member-service	8081	Tomcat started on port(s): 8081
review-service	8082	Tomcat started on port(s): 8082
api-gateway	8080	Netty started on port(s): 8080

Gateway는 Tomcat이 아니라 **Netty**에서 실행됩니다.
리액티브 기반이라 내장 서버가 다릅니다.

Step 3-1: Gateway → member-service 테스트

Postman:

```
GET http://localhost:8080/api/members/1
```

↑

포트 8080

기대 응답 (200 OK):

```
{
  "id": 1,
  "nickname": "맛집헌터",
  "email": "hunter@example.com",
  "createdAt": "...",
}
```

8080 포트로 요청했는데 member-service(8081)의 데이터가 반환됩니다.
Gateway가 요청을 member-service로 전달한 것입니다.

Step 3-2: Gateway → review-service 테스트

```
GET http://localhost:8080/api/reviews/1
```

↑

포트 8080

기대 응답 (200 OK):

```
{
  "id": 1,
  "storeName": "역전우동 강남점",
  "rating": 5,
  "content": "재방문! 역시 맛있다. 강남점도 굿!",
  "memberId": 1,
  "nickname": "맛집헌터",
  "createdAt": "...",
  "updatedAt": "...",
}
```

8080 포트로 요청했지만 review-service(8082)의 데이터가 반환됩니다.
작성자 닉네임도 포함되어 있습니다 (review-service가 내부적으로 member-service를 호출).

Step 3-3: Gateway를 통한 리뷰 작성 테스트

POST http://localhost:8080/api/reviews

Headers:

Content-Type: application/json

Body:

```
{
  "storeName": "이삭토스트",
  "rating": 4,
  "content": "아침에 간단히 먹기 좋아요. 햄치즈가 최고!",
  "memberId": 2
}
```

기대 응답 (201 Created):

```
{
  "id": 4,
  "storeName": "이삭토스트",
  "rating": 4,
  "content": "아침에 간단히 먹기 좋아요. 햄치즈가 최고!",
  "memberId": 2,
  "nickname": "먹방크리에이터",
  "createdAt": "...",
  "updatedAt": "..."
}
```

| Gateway(8080)를 통해 POST도 정상 동작합니다.

Step 3-4: 직접 호출 vs Gateway 호출 비교

같은 결과가 나오는지 비교해봅니다.

```
직접 호출: GET http://localhost:8082/api/reviews/1
Gateway:   GET http://localhost:8080/api/reviews/1
```

두 요청의 응답이 **동일**한 것을 확인하세요.

클라이언트 입장에서는 Gateway(8080)만 알면 됩니다.
뒤에 어떤 서비스가 몇 번 포트에서 동작하는지 몰라도 됩니다.

Step 4: CORS 설정 추가

CORS란?

브라우저가 **다른 출처(Origin)**의 API를 호출하면 차단하는 보안 정책입니다.

```
프론트엔드: http://localhost:3000
백엔드:     http://localhost:8080 ← 포트가 다름 = 다른 출처
```

→ 브라우저가 차단

Gateway에 CORS 설정을 추가하면, 프론트엔드에서 API를 호출할 수 있습니다.

Step 4-1: application.yaml에 CORS 추가

`application.yaml`을 수정합니다.

```
server:
  port: 8080

spring:
  application:
    name: api-gateway

cloud:
  gateway:
    server:
```

```

webflux:
  # CORS 설정 추가
  globalcors:
    cors-configurations:
      '["/**"]':
        allowedOrigins: "*"
        allowedMethods:
          - GET
          - POST
          - PUT
          - DELETE
          - OPTIONS
        allowedHeaders: "*"
        allowCredentials: false

    routes:
      - id: member-service
        uri: http://localhost:8081
        predicates:
          - Path=/api/members/**

      - id: review-service
        uri: http://localhost:8082
        predicates:
          - Path=/api/reviews/**

```

CORS 설정 해석:

항목	값	의미
<code>["/**"]</code>	모든 경로	모든 API에 CORS 적용
<code>allowedOrigins: "*" </code>	모든 출처 허용	어디서든 호출 가능
<code>allowedMethods</code>	GET, POST 등	허용할 HTTP 메서드
<code>allowedHeaders: "*" </code>	모든 헤더 허용	Content-Type 등
<code>allowCredentials: false</code>	쿠키 전송 안 함	JWT 사용 시 false로 충분

실무에서는 `allowedOrigins: "*"` 대신 프론트엔드 주소를 지정합니다.

```
allowedOrigins:
  - "http://localhost:3000"    # 개발 서버
  - "https://myapp.com"       # 운영 서버
```

Step 4-2: api-gateway 재시작

설정을 변경했으므로 api-gateway를 재시작합니다.

Step 5: CORS 동작 확인

CORS 차단 상태 먼저 체험하기

CORS가 동작하는지 확인하기 위해, 간단한 HTML 파일로 테스트합니다.

바탕화면 등 원하는 곳에 `cors-test.html` 파일을 만듭니다. (복사, 붙여넣기)

```
<!DOCTYPE html>
<html>
<head>
  <title>CORS 테스트</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      max-width: 700px;
      margin: 40px auto;
      padding: 20px;
    }
    button {
      padding: 10px 20px;
      margin: 5px;
      font-size: 16px;
      cursor: pointer;
      border: none;
      border-radius: 6px;
```

```

        color: white;
    }
    .blue { background: #2563eb; }
    .green { background: #16a34a; }
    .red { background: #dc2626; }
    pre {
        background: #1e1e1e;
        color: #d4d4d4;
        padding: 15px;
        border-radius: 8px;
        overflow-x: auto;
        min-height: 100px;
    }
    h1 { margin-bottom: 5px; }
    p { color: #666; margin-top: 0; }
</style>
</head>
<body>
    <h1>맛집 리뷰 API 테스트</h1>
    <p>Gateway(8080)를 통해 API를 호출합니다.</p>

    <button class="blue"
        onclick="callApi('/api/members/1')">
        회원 조회 (ID: 1)
    </button>
    <button class="green"
        onclick="callApi('/api/reviews')">
        리뷰 목록
    </button>
    <button class="red"
        onclick="callApi('/api/reviews/1')">
        리뷰 상세 (ID: 1)
    </button>

    <h3>응답 결과:</h3>
    <pre id="result">버튼을 클릭하세요...</pre>

```

```

<script>
  async function callApi(path) {
    const resultEl =
      document.getElementById('result');
    resultEl.textContent = '요청 중...';

    try {
      const response = await fetch(
        'http://localhost:8080' + path
      );
      const data = await response.json();
      resultEl.textContent =
        JSON.stringify(data, null, 2);
    } catch (error) {
      resultEl.textContent =
        'CORS 에러 또는 연결 실패!\n\n'
        + error.message
        + '\n\n개발자 도구(F12) > Console 탭에서'
        + '\n자세한 에러를 확인하세요.';
    }
  }
</script>
</body>
</html>

```

Step 5-1: 테스트 방법

1. `cors-test.html` 파일을 브라우저에서 더블클릭하여 엽니다.
(주소가 `file:///...` 으로 시작합니다)
(또는 라이브 서버로 열기)
2. "회원 조회" 버튼을 클릭합니다.
3. 응답 결과 영역에 회원 정보가 표시되면 **CORS 설정 성공**


```
{
  "id": 1,
  "nickname": "맛집헌터",
  "email": "hunter@example.com",
  "createdAt": "...
}
```

1. "리뷰 목록" 버튼을 클릭합니다.
2. 모든 리뷰가 작성자 닉네임과 함께 표시되면 성공입니다.

만약 에러가 발생한다면:

- 3개 서비스가 모두 실행 중인지 확인
- api-gateway를 재시작했는지 확인
- 브라우저 개발자 도구(F12) > Console 탭에서 에러 내용 확인

Step 5-2: CORS 차단 체험 (선택)

CORS 설정이 왜 필요한지 직접 체험해볼 수 있습니다.

1. api-gateway의 `application.yaml` 에서 `globalcors` 블록 전체를 주석 처리합니다.
2. api-gateway를 재시작합니다.
3. `cors-test.html` 에서 버튼을 클릭합니다.

결과: 에러 발생

브라우저 개발자 도구(F12) > Console 탭에서 아래와 같은 에러를 볼 수 있습니다:

```
Access to fetch at 'http://localhost:8080/api/members/1'
from origin 'null' has been blocked by CORS policy
```

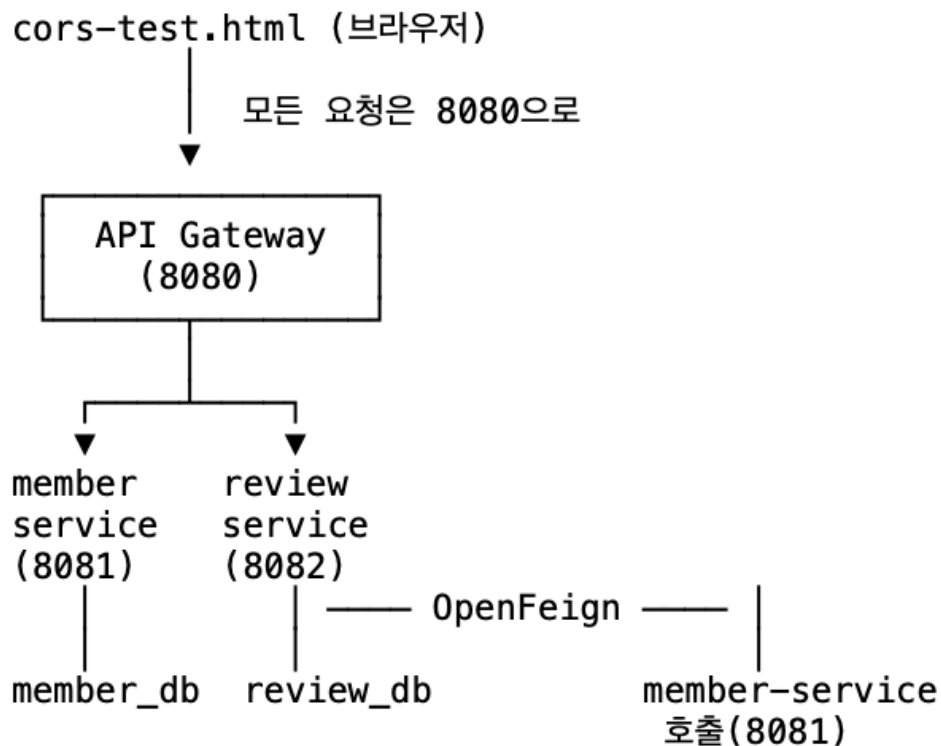
주석을 해제하고 재시작하면 다시 정상 동작합니다.

Postman에서는 CORS 에러가 발생하지 않습니다.
CORS는 **브라우저 보안 정책**이기 때문에
브라우저에서만 동작합니다.

✓ Chapter 3 체크포인트

- ☐ api-gateway가 8080에서 실행되는가?
- ☐ 8080/api/members/1 로 회원 조회가 되는가?
- ☐ 8080/api/reviews/1 로 리뷰 조회가 되는가? (닉네임 포함)
- ☐ 8080/api/reviews 로 리뷰 목록 조회가 되는가?
- ☐ HTML 파일에서 API 호출이 정상 동작하는가? (CORS)

🎉 최종 아키텍처



오늘 실습에서 경험한 것:

개념	경험
서비스 독립성	member/review 각각 독립 실행
Database per Service	member_db / review_db 분리

개념	경험
서비스 간 통신	OpenFeign으로 HTTP 호출
단일 진입점	Gateway(8080)로 통합
CORS	브라우저에서 API 호출 허용
장애 전파	member-service 중지 시 리뷰 조회 실패

추가 도전 과제 (시간이 남는다면)

도전 1: 가게 이름으로 리뷰 검색 API

ReviewRepository에 이미 `findByStoreNameContaining` 메서드가 있습니다. 이를 활용하여 가게 이름 검색 API를 추가해보세요.

```
GET /api/reviews/search?storeName=우동
```

힌트: Controller에 `@GetMapping("/search")`을 추가하고, `@RequestParam`으로 `storeName`을 받으세요.

도전 2: 특정 회원의 리뷰 목록 API

ReviewRepository에 `findByMemberId` 메서드도 있습니다. 특정 회원이 작성한 리뷰만 조회하는 API를 추가해보세요.

```
GET /api/reviews/member/1
```

도전 3: 회원 정보 수정 API

member-service에 회원 정보 수정(PUT) API를 추가해보세요. 닉네임과 이메일을 변경할 수 있도록 합니다.

```
PUT /api/members/1
Body: { "nickname": "미식가", "email": "new@example.com" }
```

힌트: Member Entity에 `update` 메서드를 추가하고, Service → Controller 순서로 구현합니다.

제출

msa-project-practice 폴더를 생성하고 그 안에 아래와 같이 구현한 프로젝트를 모두 이동
msa-project-practice를 레포지토리로 만들고 깃허브 클래스룸으로 제출

```
msa-project-practice/  
├─ member-service/  
├─ review-service/  
└─ api-gateway/
```